

① 1) What is object
object is something which has 1) state, 2) behaviour,
3) Identity, 4) Responsibility.

Exa:- Mobile.

2) What is state

State means giving value to the attribute
It is attribute + value.

Exa:- OS - Android, Color - black
Storage - 128 GB, Brand - Vivo
Network type - 5G, Price - 20,000

3) What is behaviour.

Response given by object to outside the world.

Exa:- When I press the power button, then the mobile turns on.

4) What is identity

Identity means uniqueness associated with particular object.

To differentiate ^{the object} between same state & behaviour.

Exa:- IMEI Number :- 123456789012345.

5) What is Responsibility :-

role of an object in real world / programming.

Exa:- To access the internet

To provide communication

To make & receive calls.

To store and provide information.

② What is object?

Object is something which has 1) State, 2) behaviour, 3) Identity, 4) Responsibility

Exa:- Umbrella.

What is state?

State is giving value to the attribute.

It is attribute + value.

Exa:- Color - Red.

What is behaviour?

Response given by object to outside the world.

Exa:- When I press the button, then the umbrella opens.

What is identity?

Identity means uniqueness associated with particular object.

To differentiate between same state & behaviour.

exa:- Serial Number - UMB101.

What is Responsibility?

Role of an object in real world / Programming.

exa:- To Protect from rain

③ What is object?

Object is something which has 1) state, 2) behaviour, 3) identity, 4) Responsibility.

Exa:- Bag.

What is state?

State is given value to the Attribute.

Exa:- Color - black, Brand - Lenovo, Price = ₹1200

Number of Compartments : 3.

What is behaviour?

Response given by object to outside the world.

Exa:- When I pull the chain, then bag opens.

What is identity?

Identity means uniqueness associated with particular object.

Exa:- Bag Serial Number : BA6101

To differentiate the object between same state and behaviour.

What is Responsibility?

Role of an object in real world, programming.

Exa:- To store academic equipments,
To carry and protect the laptop.

④ What is object?

object something which has

① state

② behaviour

③ Identity

④ Responsibility

for exa:- notebook

What is state?

State is given value to the attribute.

price = 86₹, Color - White, Type of Ruling - Grid

line, Size = 29.7 x 21 cm.

What is behaviour

Response given by object to outside the world.

Exa :- ① When I turn a page, then the notebook shows the next page.

② When I write on the notebook, then the notebook records the information.

What is Identity

Identity means uniqueness associated with particular object.

Exa :- ① Notebook ID = NB101.

To differentiate the object between same state and behaviour.

What is Responsibility.

Responsibility - Role of an object in real world.
To store and write notes.

⑤ What is object

Object something which has

① state

② behaviour

③ identity

④ Responsibility

exa :- Water Bottle.

What is state?

State is given value to the attribute

price :- 200, color :- blue.

What is behaviour?

Response gives by the object to outside the world.

Exa:- When I open the bottle cap, then I can drink water.

What is identity?

Identity means uniqueness associated particular object.

Exa:- Unique Bottle ID - WB101.

To differentiate the object between same state of behaviour.

Exa:-

What is Responsibility?

Role of an object in real world.

To store water and quench thirst.

⑤ What is Abstraction?

- Abstraction is process of selective ignorance.
- Abstraction is process of Requirement gathering which is present at software development life cycle.
- It is completely pen and paper process.
- We can not write a single line of code for a Abstraction.
- Note :- Abstraction and Abstract class is a different class.

⑥ Encapsulation

Encapsulation is the process of binding and hiding states and behaviour.

- Why there is need to bind states and behaviour.

→ They are inseparable in real life

Note :- Behaviour / function are public in real life and states are private in real life.

- How we can implement Encapsulation?

→ by using class we can implement encapsulation.

- What is mean by class.

Class is a way implementing encapsulation.

Abstraction Example

classmate

Date

Page

1

Vehicle	Car	Bike
Name	Name, engine	Name
Color	Color, Mileage	Color
Weight	Weight, Battery	Weight
Speed	Speed	Speed
Fuel-type	Fuel-type	Fuel-type
No. of wheels	No. of wheels	No. of wheels
Seating-capacity	Seating-capacity	Handle type
AC	AC	Helmet Required
Boot-space	Boot-Space	Gear-type
Handle-type	Gear type	Price
Helmet-required	Price	
Gear-type		
Price		

2

Savings Account	Bank Account	Current Account
Account Number	Account Number	Account Number
Account Holder Name	Account Holder Name	Account Holder Name
Address	Address	Address
Holder Mobile Number	Mobile Number	Mobile Number
Balance	Balance	Balance
IFSC code	IFSC Code	IFSC Code
Interest Rate	Interest Rate	Minimum Balance
ATM card	ATM card	Business Name
Passbook	Passbook	Inst Number
Minimum Balance	Minimum Balance	Transaction Limit.
Transaction Limit	Business Name	
	Inst Number	
	Transaction limit	

3	Teacher	School Member	Student
	Age, Name, Gender, Address, Mobile Number, Qualification, Subject, Salary, Employee ID.	Name, Age, Gender, Address, Mobile Number, Email, Qualification, Salary, Subject, Employee ID, Roll No, Class, Division, Fees	Name, Age, Gender, Address, Mobile Number, Email, Roll No, Class, Division, Fees.

4	Mobile	Product	Laptop
	Product Name, Brand, Price, Color, Weight, Warranty, Model No, RAM, Storage, Screen Size, Camera, Battery, OS	Product Name, Brand, Price, color, Weight, Warranty, Model No, RAM, Storage, Processor, Screen Size, Camera, Battery, OS, Keyboard	Product, Name, Brand, price, color, Weight, Warranty, model No, RAM, storage, processor, screen size, camera, Battery, Operating system, Keyboard (physical)

5	TV	Electronic Device	Refrigerator (Fridge)
	Brand,	Brand,	Brand, Model No.
	Model No.,	Model No.,	Price, color,
	Price,	Price, Color,	Weight,
	Color,	Weight, Power Consumption,	power consumption
	Weight,	Warranty	Warranty
	Power Consumption,	Display Size,	Number of Doors
	Warranty,	Resolution,	Capacity,
	Display Size,	Speaker,	Freezer,
	Resolution,	Number of Doors,	Cooling Type
	Speaker.	Capacity, Freezer,	
		Cooling Type	

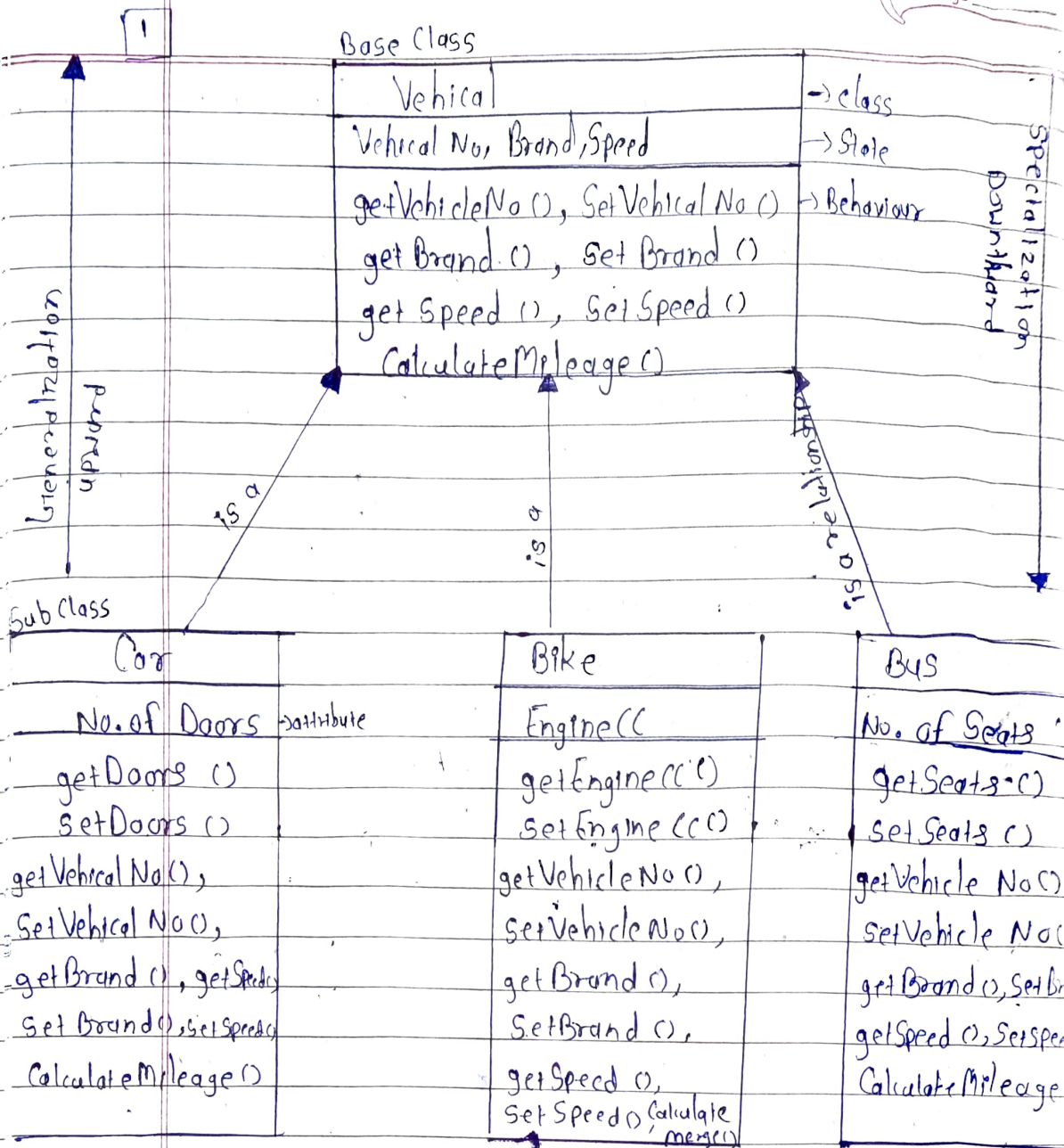
6	Text Book	Book	Story Book
	Book Name	Book Name	Book Name
	Author	Author	Author
	Price	Price	Price
	Publisher	Publisher	Publisher
	Pages	Pages	Pages
	Language	Language	Language
	ISBN	ISBN	ISBN
	Subject	Subject	Story Type
	Course	Course	Characters
	Semester	Semester	Genre
	Chapter	Chapter	Publication Year
	Publication Year	Story type	
		Characters	
		Genre	
		Publication Year	

Inheritance Example

classmate

Date

Page



② Reuse already defined classes with "is-a-relationship."

Why there is need to reuse?

To reduce the code redundancy,
by removing.

If there is code redundancy in your
program So there is high-chances
of system failure

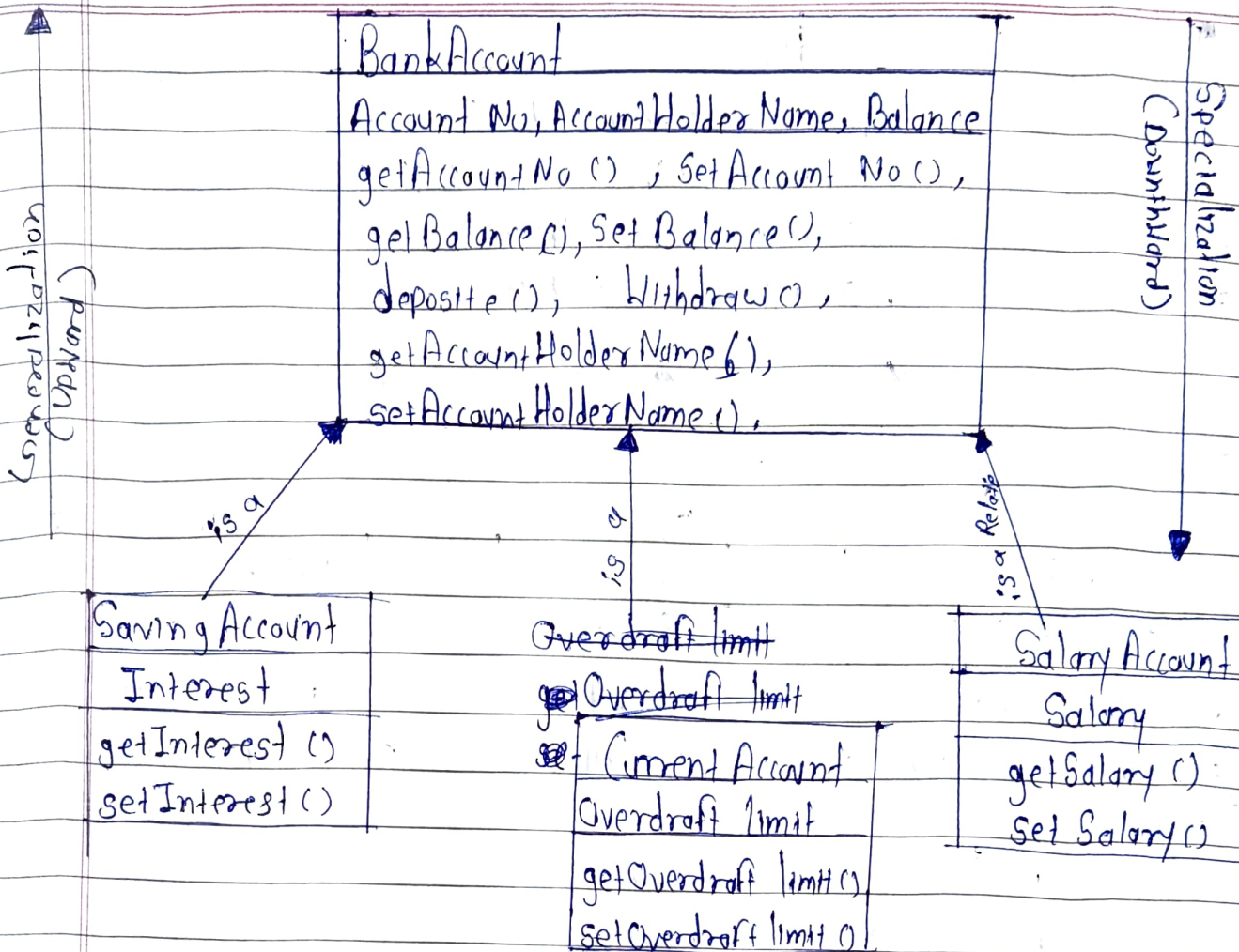
get = retrieve/read
set = set/update

classmate

Date _____

Page _____

2

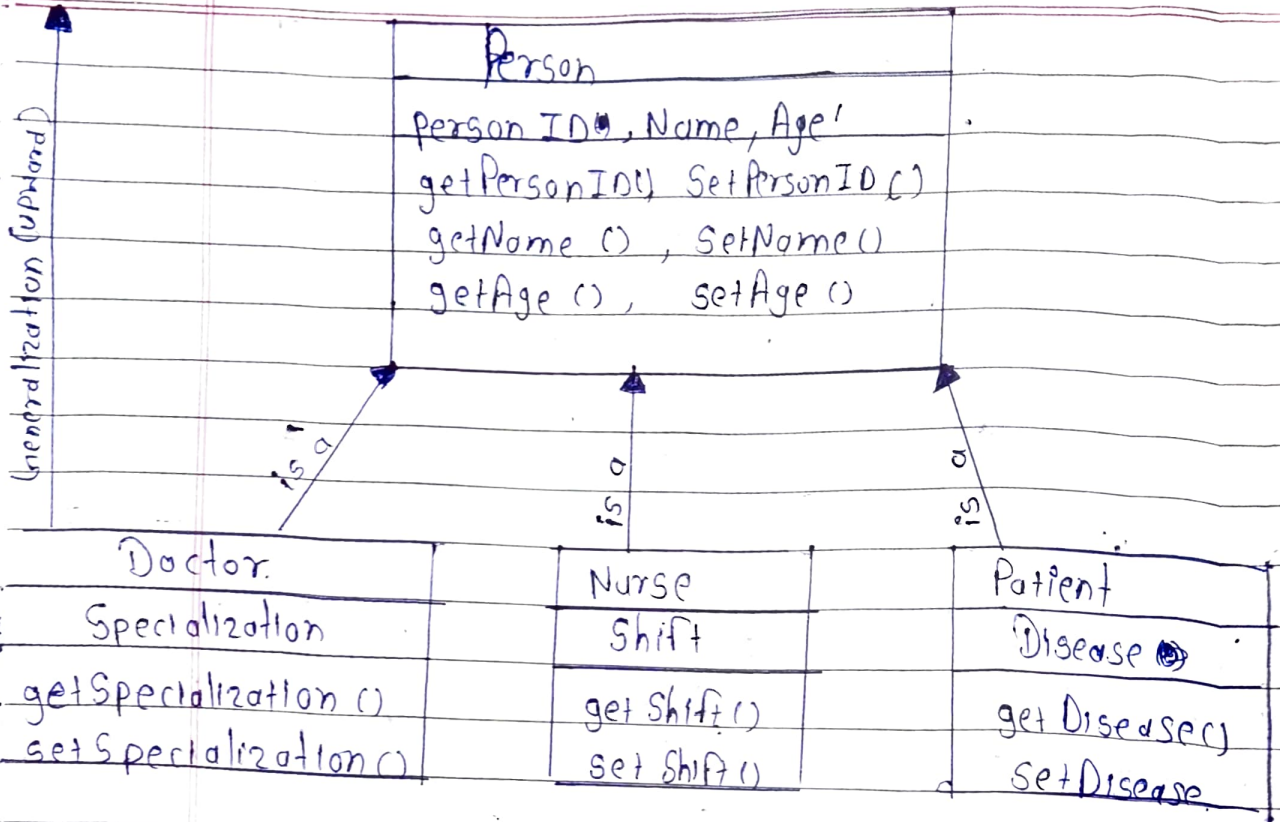


3

- Reuse already defined classes with "is-a-relationship".
- Why there is need to reuse?
- To reduce the code redundancy, by removing
- If there is code redundancy in your program
So there is high chances of system failure

upward to the hierarchy We get Generalization
downward to the hierarchy We get Specialization.

3



Doctor is a person

Nurse is a person

Patient is a person

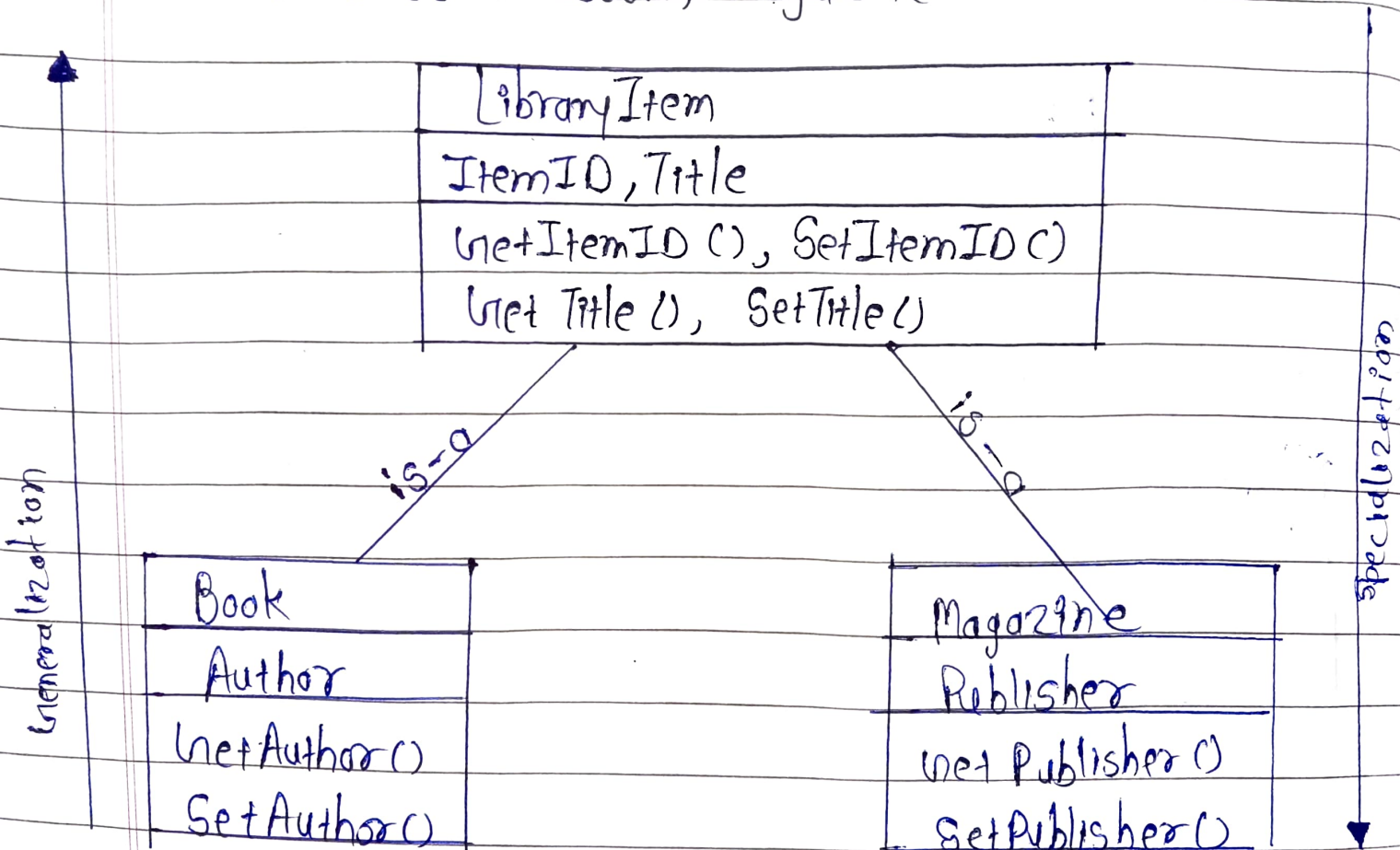
[4] Reuse already defined classes with "is-a-Relationship".

- Why there is need to reuse.
To reduce the code redundancy, by removing
- If there is code redundancy in your program, So there is high chances to system failure.

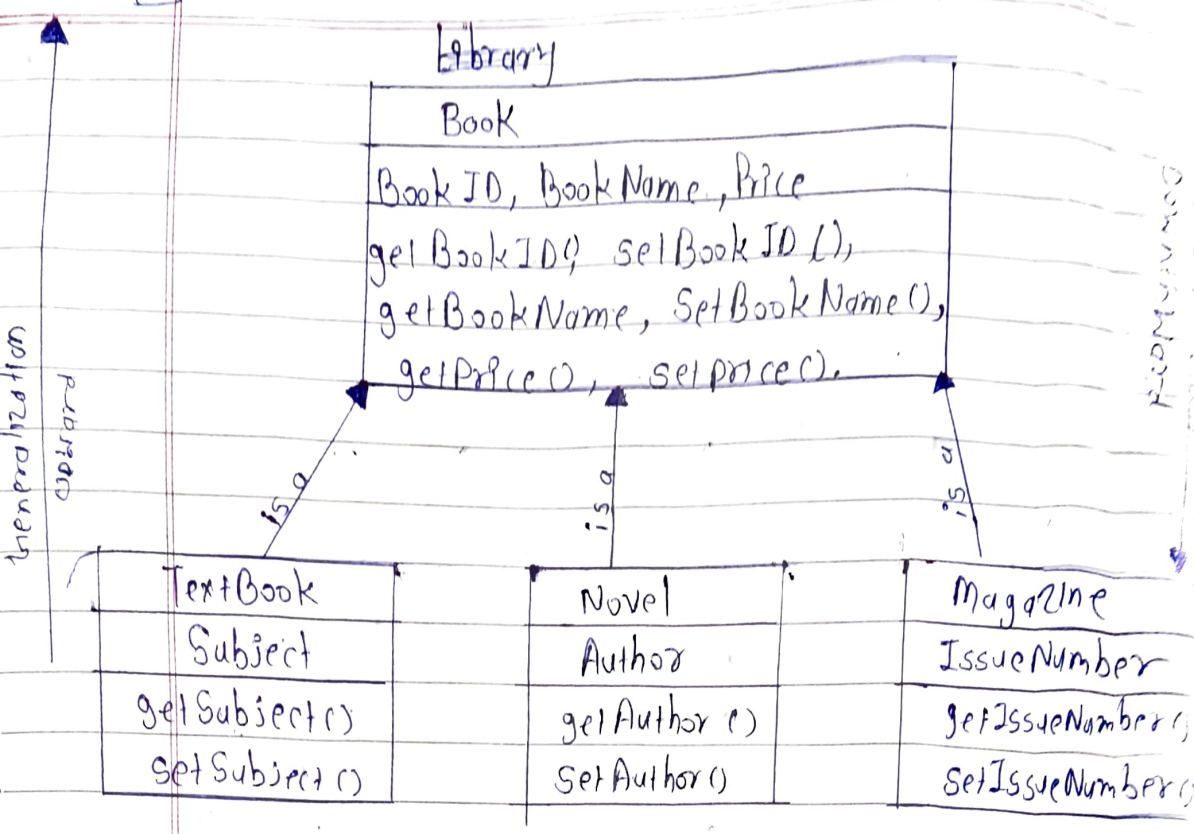
upward to the hierarchy we get generalization.
downward to the hierarchy we get specialization.

④

Base class - LibraryItem
Derived class - Book, Magazine



5



upward to the hierarchy we get generalization ^{SD. General}
 downward to the hierarchy we get specialization. ^{SD. Specific}

IS-a-Relationship

Has-a-Relationship

It represents Inheritance

It represents Association / Composition.

It shows a parent-child relationship between classes.

It shows a whole-part relationship between classes.

It means "one class is a type of another class."

It means "one class contains or uses another class."

It is achieved using Inheritance.

It is achieved by using objects of a class.

Compile time polymorphism is done by the process of name mangling.

* Polymorphism Examples *

1 Example :- ① Payment.

1. Same message - Pay()
2. Given to generalize things → Payment Methods.
3. For same behaviour → Making Payment
4. But implemented differently → Different Payment Methods make payment differently.

for example:- UPI → UPI use

CreditCard → Card details use for payment

Cash - Cash payment

Same message given to generalize things for same behaviour but implemented differently.

②

login

Same Message — login()

Generalized Thing — login system

Same Behaviour :- Login into the system.

But implemented differently

Google :- Login With Google

Facebook :- Login With Facebook

Email :- Login With Email.

[3] Example :- Printer

1. Same message :- print()
2. Given to generalize things → printers
3. For same behaviour → printing the document.
4. But implemented differently → Different printers print the document using different methods. exa:-
 - Laser printer → Toner use
 - Inkjet printer → Ink use
 - 3D Printer → Material Object use.

[4] Example :- Passenger

1. Same message :- travel()
2. Given to generalize things :- passenger
3. For same behaviour :- Traveling.
4. But implemented differently → Different passengers travel by different modes of transport.

for exa:-

Bus Passenger → Bus travel
 Train Passenger → Train travel
 Air Passenger → Aeroplane travel.

[5] Example :- Shape

1. Same message - draw()
2. Given to generalize things - Shapes.
3. For same behaviour → Drawing a shape.
4. But implemented differently → Different shapes are drawn differently.

for exa:- Circle → Circle draw
 Rectangle → Rectangle draw
 Triangle → Triangle draw

* Abstraction Example *

```

1 class Vehicle :
    def __init__(self, VehicleNo, Brand, Speed):
        self.VehicleNo = VehicleNo
        self.Brand = Brand
        self.Speed = Speed
    print ("This is a constructor")
    def display(self):
        print (f"VehicleNo = {self.VehicleNo},
                Brand = {self.Brand},
                Speed = {self.Speed}")
V1 = Vehicle ("MH12AB1234", "Tata", 70)
print (id (V1))
V1.display ()

```

Output :- It is a Constructor

1452189058448

VehicleNo := MH12AB1234, Brand = Tata,
speed = 70.

```

2 class BankAccount:
    def __init__(self, AccountNo, AccountHolderName, Address):
        self.AccountNo = AccountNo
        self.AccountHolderName = AccountHolderName
        self.Address = Address
    # print ("It is constructor")
    def display(self):
        print (f"AccountNo = {self.AccountNo},
                AccountHolderName = {self.AccountHolderName},
                Address = {self.Address}")
BA = BankAccount ("SBI NO 0001234", "Ankita", "Gurur")
print (id (BA))
BA.display ()

```

Output:- 2448863495568

AccountNo = SBIN00001234, AccountHolderName = "Anshu",
Address = Latim.

```
(3) Class Product :
    def __init__(self, ProductName, Color, Warranty):
        self.ProductName = ProductName
        self.Color = Color
        self.Warranty = Warranty
    def print("This is a constructor")
    def display (self):
        print(f"ProductName = {self.ProductName},
              Color = {self.Color},
              Warranty = {self.Warranty}.")
P1 = Product ("lenovo", "Platinum Grey", "3 years")
Print (id (P2))
P2 . display ()
```

Output:- This is a constructor
2164435813776

ProductName = lenovo, Color = Platinum Grey, Warranty =
3 years.

```
(4) Class Book :
    def __init__(self, BookName, Author, Price)
        self.BookName = BookName
        self.Author = Author
        self.Price = Price
    print (" This is a constructor")
    def display (self)
B = Book print (f" BookName, BookName = {self.
BookName}, Author = {self.Author}, Price = {self.
Price})
```



```
B = Book ("Gandhep", "Bharat Aandhale", 150)
print (id (B))
B.display ()
```

Output :-

This is a Constructor

1997005161872

BookName = Gandhep , Author = Bharat Aandhale ,
Price = 150.

⑤ Class Student :

```
def __init__ (self, id, name, klass)
```

```
    self.id = id
```

```
    self.name = name
```

```
    self.class = klass
```

```
    print ("That is the constructor")
```

```
def display (self)
```

```
    print (f" id = {self.id}, name = {self.name},  
           class = {self.class} ")
```

```
#object s1 = Student (101, "Ankita Shinde", "MCA FY")
```

```
print (id (s1))
```

```
s1.display ()
```

Output - That is the constructor

1997359056272

id = 101, Name = Ankita Shinde, class = MCA FY.

* Encapsulation Example *

classmate

Date _____

Page _____

```
① Class Vehicle :  
    def __init__(self, VehicalNo, Brand, Speed):  
        self.VehicalNo = VehicalNo  
        self.Brand = Brand  
        self.speed = speed  
  
    def getVehicalNo (self):  
        return self.VehicalNo  
    def setVehicalNo (self, newId):  
        self.VehicalNo = newId  
    def getBrand (self):  
        return self.Brand  
    def setBrand (self, newId):  
        self.Brand = newId  
    def getspeed (self):  
        return self.speed  
    def setspeed (self, newId):  
        self.speed = newId  
    def display (self):  
        print (f "VehicalNo = {self.VehicalNo},  
                Brand = {self.Brand}, speed = {self.speed}")  
  
v1 = Vehicle ("MH12AB1234", "Tata", 70)  
v2 = Vehicle ("MH13CD1234", "MarutiSuzuki", 90)  
print (v1.getSpeed ())  
v1.setSpeed (90)  
print (v1.getSpeed ())
```

Output:

70

90

```

2) class BankAccount:
    def __init__(self, AccountNo, AccountHolderName, Address):
        self.AccountNo = AccountNo
        self.AccountHolderName = AccountHolderName
        self.Address = Address
    def getAccountNo (self):
        return self.AccountNo
    def setAccountNo (self, AccountNo):
        self.AccountNo = AccountNo
    def getAccountHolderName (self):
        return self.AccountHolderName
    def setAccountHolderName (self, AccountHolderName):
        self.AccountHolderName = AccountHolderName
    def getAddress (self):
        return self.Address
    def setAddress (self, Address):
        self.Address = Address
    def display (self):
        print (f"AccountNo = {self.AccountNo} ,
        AccountHolderName = {self.AccountHolderName},
        Address = {self.Address} ")

BA = BankAccount ("SBI 00001234", "Ankita", "Latur")
BA2 = BankAccount ("HDFC 0001235", "Vaishnavi", "Kosarkote")
print (BA.getAddress (1))
BA.setAddress ("Mumbai")
print (BA.getAddress (2))

```

Output:- Latur
Mumbai.


```
⑤ class Book:
    def __init__(self, BookName, Author, Price):
        self.BookName = BookName
        self.Author = Author
        self.Price = Price
    def getBookName(self):
        return self.BookName
    def setBookName(self, BookName):
        self.BookName = BookName
    def getAuthor(self):
        return self.Author
    def setAuthor(self, Author):
        self.Author = Author
    def getPrice(self):
        return self.Price
    def setPrice(self, Price):
        self.Price = Price
    def display(self):
        print(f"BookName = {self.BookName},  

            Author = {self.Author}, Price = {self.Price}")
B = Book("Namdeep", "Bharat Aandhale", 150)
B2 = Book("Murtaniday", "Shivaji Savant", 500)
print(B.getPrice())
B.setPrice(200)
print(B.getPrice())
Output:-      150
                200
```

④

def 1

class Product:

def __init__(self, ProductName, Color, Warranty):

self.ProductName = ProductName

self.Color = Color

self.Warranty = Warranty

def getProductName(self):

return self.ProductName

def setProductName(self, ProductName):

self.ProductName = ProductName

def getColor(self):

return self.Color

def setColor(self, Color):

self.Color = Color

def getWarranty(self):

return self.Warranty

def setWarranty(self, Warranty):

self.Warranty = Warranty

def display(self):

print(f"ProductName = {self.ProductName},
Color = {self.Color}, Warranty =
{self.Warranty}")

P1 = Product("Lenovo", "Platinum Grey", "3 years")

P2 = Product("HP", "White", "2 years")

print(P2.getProductName())

P2.setProductName("Dell")

print(P2.getProductNames())

Output:

HP

Dell

(5)

Class Student:

```
def __init__(self, id, name, class):  
    self.id = id  
    self.name = name  
    self.class = class  
def getid (self):  
    return self.id  
def setid (self, id):  
    self.id = id  
def getname (self):  
    return self.name  
def setname (self, name):  
    self.name = name  
def getclass (self):  
    return self.class  
def setclass (self, class):  
    self.class = class  
def display (self):  
    print (f"id={self.id}, Name={self.name}, class=  
          {self.class}")
```

```
s1 = Student (101, "Ankita Shinde", "MCA 5y")
```

```
s2 = Student (102, "Vaishnavi", "8th")
```

```
print (s1.getid ())
```

```
s1.setid (100)
```

```
print (s2.getid ())
```

Output:- 101

 100